

Методические указания к лабораторной работе №2

Перегрузка операций в классах.

Перегрузка стандартных операндов языка C# для объектов позволяет наделить их «пониманием» операций, применяемых для элементарных типов данных.

Как правило, перегрузка операций присуща объектам классов, являющихся сущностями-значениями, специфичными для предметной области. Например, комплексные числа, рациональные дроби, вектора, матрицы и т.д.

Рассмотрим пример рациональной дроби. Дробь состоит из целочисленного числителя и натурального знаменателя. Стандартного типа дроби нет, хотя он бы обеспечил хранение и обработку рациональных чисел без потери точности на преобразовании в десятичную вещественную дробь. Пример, число $1/3$ в double равно 0.33333..., т.е. приближенное. Ситуация усугубляется еще и тем, что вещественные дроби хранятся в двоичной системе счисления, т.е. присутствует дополнительная потеря точности в сравнении с десятичной системой.

```
public class RationalFraction
{
    private readonly int _numerator;
    private readonly int _denominator;

    public RationalFraction(int numerator = 0, int denominator = 1)
    {
        if (denominator == 0) throw new DivideByZeroException();

        if (denominator < 0)
        {
            denominator *= -1;
            numerator *= -1;
        }

        _numerator = numerator;
        _denominator = denominator;
    }

    public double Value
    {
        get { return (double) _numerator/_denominator; }
    }

    public override string ToString()
    {
        return $"{_numerator}/{_denominator}";
    }
}
```

Объект рациональной дроби или ее значение сделан неизменяемым, чтобы избежать ошибок, связанных с множеством ссылок на один объект. Подобным образом реализован тип string.

Дробь хранит свое значение в двух полях: числитель и знаменатель. Свойство Value нужно для преобразования в double. Переопределенный из класса object метод ToString() преобразует значение дроби в строку для вывода на консоль или файл.

В C# нельзя перегружать оператор присваивания, но есть возможность запрограммировать логику неявного (implicit) преобразования типов данных. Добавим преобразование из int в дробь и из дроби в double.

```
public static implicit operator RationalFraction(int value)
{
    return new RationalFraction(value);
}

public static implicit operator double(RationalFraction fraction)
{
    return fraction.Value;
}
```

Далее перегрузим операции.

```
public static RationalFraction operator +(RationalFraction v1, RationalFraction v2)
{
    return new RationalFraction(
        v1._numerator*v2._denominator + v2._numerator*v1._denominator,
        v1._denominator*v2._denominator);
}

public static RationalFraction operator -(RationalFraction v1)
{
    return new RationalFraction(-v1._numerator, v1._denominator);
}

public static RationalFraction operator -(RationalFraction v1, RationalFraction v2)
{
    return v1 + (-v2);
}

public static RationalFraction operator *(RationalFraction v1, RationalFraction v2)
{
    return new RationalFraction(
        v1._numerator*v2._numerator, v1._denominator*v2._denominator);
}

public static RationalFraction operator /(RationalFraction v1, RationalFraction v2)
{
    if (v2._numerator == 0) throw new DivideByZeroException();

    return new RationalFraction(v1._numerator*v2._denominator, v1._denominator*v2._numerator);
}
}
```

Класс готов, добавлена перегрузка операции сложения (+), унарной операции смены знака (-), вычитания (-), реализованной через сложение и смену знака, умножения (*) и деления (/).

Для перегрузки логично выбирать ту операцию, которая максимально соответствует алгоритму перегрузки. Арность операции сохраняется. Составные операции, например, (+=) поддерживаются компилятором автоматически из перегруженной операции (+).

Результатом каждой перегруженной операции является новый объект, новая дробь, т.к. сами объекты не изменяемы по значению.

Примеры работы с рациональными дробями.

```
var a = new RationalFraction(1, 3); // 1/3
var b = new RationalFraction(2, 3); // 2/3

var c = a + b;
Console.WriteLine($"c = {c.Value}"); // 1

double d = (b - a)*3;
Console.WriteLine($"d = {d}"); // 1

var e = (a += 1) - 1;
Console.WriteLine($"a = {a}, e = {e}"); // 4/3, 1/3

var f = new RationalFraction(1)/3;
Console.WriteLine($"f = {f}"); // 1/3
Console.WriteLine(f); // 0.3333333
```

Создадим две дроби, 1/3 и 2/3. Переменная c – это тоже дробь, результат сложения a и b. Переменная d – double, задействовано неявное преобразование типов из дроби в вещественный тип. Умножение на 3 результата b - a работает, потому что 3 неявно из int преобразуется к дроби.

Если вывести дробь f на консоль, то будет задействовано неявное преобразование сначала в double, а затем уже в string, потому что для метода WriteLine() есть перегруженный вариант с аргументом типа double. Если же f сразу включить внутри строки, то будет вызван метод ToString() для дроби.

В следующем примере реализован класс безопасной парольной строки на основе массива символов. Обычные строки (string) неопределённое время после окончания использования находятся в оперативной памяти, это не безопасно для паролей, которые могут попасть в дампы памяти во время сохранения его на жесткий диск. Для безопасной строки реализован метод Dispose() интерфейса IDisposable, обнуляющий внутренний массив символов строки в конце ее жизненного цикла.

```
public sealed class SecurityString : IDisposable
{
    private char[] _value;

    private SecurityString(char[] value) { _value = value; }

    public SecurityString()
    {
        _value = new char[0];
    }

    public SecurityString(string value)
    {
        _value = value.ToCharArray();
    }

    public int Length { get { return _value.Length; } }
```

Класс закрыт (sealed) для наследования, содержит единственное поле внутреннего массива `_value`, вспомогательный приватный и два публичных конструктора для инициализации объектов.

Класс не имеет методов просмотра внутреннего содержимого массива, вместо этого присутствует метод и перегруженные операции сравнения строки с другими объектами для проверки пароля.

Дополнительные методы класса: добавление к строке символа (`AppendChar`), возврат подстроки (`Substring`), сравнение строк (`Compare`).

```
public SecurityString AppendChar(char ch)
{
    char[] value = new char[_value.Length + 1];

    Array.Copy(_value, value, _value.Length);
    Array.Clear(_value, 0, _value.Length);

    value[_value.Length] = ch;

    _value = value;
    return this;
}

public SecurityString Substring(int index, int length = 0)
{
    if (length == 0) length = _value.Length - index;

    char[] value = new char[length];
    Array.Copy(_value, index, value, 0, length);

    return new SecurityString(value);
}

public bool Compare(SecurityString value, int index, int length = 0)
{
    if (length == 0) length = value.Length;

    if (index < 0
        || length > value.Length
        || index + length > _value.Length) return false;

    for (int i = 0; i < length; i++)
        if (_value[i + index] != value._value[i]) return false;

    return true;
}
```

Перегруженные методы класса `object` и интерфейса `IDisposable`:

```
public override bool Equals(object obj)
{
    SecurityString value = obj as SecurityString;

    return value?.Length == _value.Length
        && Compare(value, 0);
}
```

```

public override int GetHashCode()
{
    int hash = 0;

    for (int i = 0; i < _value.Length; i++)
        hash ^= (hash << 5) + (hash >> 2) + _value[i];

    return hash;
}

public void Dispose()
{
    if (_value != null)
    {
        Array.Clear(_value, 0, _value.Length);
        _value = null;
    }
}

```

Метод `Array.Clear()` нужен для обнуления памяти массива. Метод вычисления хеша строки (`GetHashCode`) позволяет получить относительно уникальное число для каждой строки. Чтобы не хранить правильные пароли в явном виде, обычно хранят их хеши и сравнивают значения хешей.

Перегруженные операции над строками:

```

public static SecurityString operator +(SecurityString value, char ch)
{
    return value.AppendChar(ch);
}

public static bool operator ==(SecurityString v1, SecurityString v2)
{
    if ((object) v1 == null
        || (object) v2 == null) return false;

    return v1.Equals(v2);
}

public static bool operator !=(SecurityString v1, SecurityString v2)
{
    return !(v1 == v2);
}

public SecurityString this[int start, int finish]
{
    get { return Substring(start, finish - start + 1); }
}

```

Данные операции в основе своей используют методы класса, поэтому не обязательны, но, возможно, с ними более удобно работать.

Перегрузка (+) особенна тем, что в результате не создается новая строка, символ добавляется к существующему массиву, поэтому дублирования данных в памяти нет. Удобно использовать не как (+), а как (+=).

Перегрузка [] или индексатор (this) реализован с двумя параметрами для возврата подстроки исходной строки, например, чтобы сравнивать части парольной строки.

Пример использования безопасной строки в консольном приложении. Программа должна запросить пароль пользователя без отображения вводимых символов на экране и сравнить пароль с эталоном (хешем). Чтобы парольная строка являлась таковой, и можно было ее отличить от других строк по содержанию, в начале у нее будет стандартный текст или токен.

```
const string passToken = "my pass:";

int correctHash = new SecurityString("BGeE5JgT").GetHashCode();
```

Далее реализован цикл посимвольного ввода пароля с отображением только звездочек.

```
var strPass = new SecurityString(passToken);
Console.Write("Enter your password: ");

ConsoleKeyInfo key;
do
{
    key = Console.ReadKey(true);

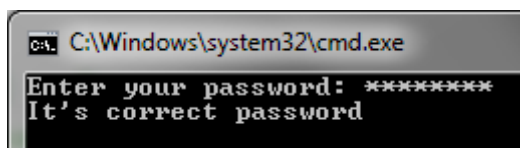
    if (key.Key != ConsoleKey.Backspace
        && key.Key != ConsoleKey.Enter)
    {
        strPass += key.KeyChar;
        Console.Write("*");
    }
}
while (key.Key != ConsoleKey.Enter);
Console.WriteLine();
```

Каждый введенный символ пароля добавляется к парольной строке перегруженной операцией (+=).

Проверка пароля осуществляется по наличию токена в начале строки и сравнению хешей.

```
if (strPass.Length > passToken.Length)
{
    using (var token = strPass[0, passToken.Length - 1])
    using (var pass = strPass[passToken.Length, strPass.Length - 1])
    {
        if (token == new SecurityString(passToken)
            && pass.GetHashCode() == correctHash)
        {
            Console.WriteLine("It's correct password");
        }
    }
}
else Console.WriteLine("Empty password");
strPass.Dispose();
```

Важно для каждой строки вызвать метод Dispose() либо явно, либо неявно в конце блока using. Пример работы программы:



```

C:\Windows\system32\cmd.exe
Enter your password: *****
It's correct password
```